

16. Bölüm

Dosyalar, Akışlar ve Serileştirme

16.1 Akış Nedir?

C# dilinde akış (**stream**), verilerin bir kaynaktan (klavye, dosya, ağ) bir hedefe (ekran, dosya, bellek) doğru aktığı **soyut bir "kanal" veya "boru hattı"** olarak düşünülür. Akışlar, verinin fiziksel olarak **nerede saklandığıyla ilgilenmez**; sadece **verinin sıralı bir şekilde transfer edilmesini sağlar**. Bu nedenle **akış (stream)**, verilerin bir kaynaktan bir hedefe doğru **sıralı ve sürekli bir şekilde hareket etmesini** temsil eden soyut bir kavramdır.

Bir de kontrol akışı (**flow**) kavramı vardır. **Akış Kontrolü (control flow)**, kontrol işlemlerinin anlatıldığı 6.1 başlığında işlenmiştir. Kodun icra sırası anlamındadır.

16.2 Dosyalar ve Klasörler

Kısaca dosya (**file**), verilerin bir araya geldiği (**collection of data**) ikincil saklama ortamıdır (**secondary storage**). Bu ikincil ortam; Bir **bilgisayar belleği (memory)** olabileceği gibi, verilerin elektrikler kesildiğinde kaybolmayacağı **sabit disk (hard disk)**, ya da bir başka ortama/bilgisayara veri gönderen (**send**) veya alan (**receive**) modem ve benzeri bir **cihaz (device)** olabilir. **Akışları (stream)** temsil eden boru hattının iki ucuna bağlanabilen yapılardır.

Dosyalar söz konusu olduğunda bazı kavramları bilmemiz gerekir;

- ♦ **Yol (path)**: Dosyanın dosya sistemindeki konumu. Örneğin; `C:\Users\ILHANOZKAN\Okubeni.txt`. Burada yolun `C:\Users\ILHANOZKAN\` kısmı dosyanın içinde bulunduğu klasörü (**directory**) gösterir. Geri kalan `Okubeni.txt` ise dosyanın adı ve uzantısını temsil eder. **Tam metin (verbatim string)** kullanarak dosya yolundaki ters bölü karakterini iki kez yazmaktan kurtulabiliriz.
- ♦ **Sonuna Ekleme (append)**: Dosya mevcutsa, dosyanın sonuna veri ekler değilse yeni dosya üzerine yazar.
- ♦ **Metin (text)**: Yazılacak veya saklanacak verinin metin şeklinde, yani gözle okunabilir olduğunu ifade eder. Yani okunup yazılan veri, basit metin editörleriyle görülebilir.
- ♦ **İçerik (contents)**: Yazılacak metinlerin bir koleksiyonudur.
- ♦ **Kaynak (source)**: Kaynak dosyanın dosya sistemindeki konumu.
- ♦ **Hedef (destination)**: Hedef dosyanın dosya sistemindeki konumu

Veriler dosyalara aynı nehirde birbirinin peşi sıra giden tekneler gibi bayt-bayt gönderilir veya alınırlar. Bu nedenle dosyalara veri taşıyan ve veri getiren nesnelere **akış (stream)** adı verilir. Benzer şekilde nehirde hangi sırada veri gelirse o sırada verileri nehirde alırız. Burada nehir akış tanımındaki boru hattıdır. Bu boru hattına konulacak nesneler, art arda gönderilecek belli sırada baytlara çevrilir. Bu işleme **serileştirme (serialization)** adı verilir. Boru hattının diğer ucunda bu nesneyi oluşturacak bu baytları bir araya getirme işlemine ise **seriden çıkarma (deserialization)** adı verilir.

16.3 Veri Akışları

C# dilinde akış işlemleri `System.IO.Stream` kütüphanesinde altında toplanırlar. En temel anlamda akış, bir kaynaktaki (dosya, ağ bağlantısı, bellek) baytları okumak veya yazmak için kullanılan mekanizmadır. Verinin tamamını bir anda belleğe yüklemek yerine, küçük parçalar halinde işlenmesine olanak tanır.

1. **MemoryStream**: Verileri bellek (RAM) üzerinde bir akış gibi yönetmek için.
2. **FileStream**: Dosyalardan veri okumak/yazmak için.
3. **NetworkStream**: İnternet veya yerel ağ üzerinden veri transferi için.

Bir ağ akışı (**NetworkStream**) üzerinden veri okurken internet kablosu çekilirse ne olur? Akış "bozulur" ve program `IOException` fırlatır. Akışlar genellikle "bağlantı temelli" (**connection-oriented**) oldukları için, kaynak ile hedef arasındaki yolun açık kalması gerekir. Bu yüzden tüm akışlar, kullanıldıktan sonra mutlaka **Close()** veya **Dispose()** ile kapatılmalıdır. Ayrıca akışları açık tutmak maliyetlidir. Bu nedenle işimiz bitince hemen kapatılmalıdır.

16.4 Dosya İşlemleri

C#, `System.IO` isim alanı altında çok daha zengin ve yönetimi kolay bir araç seti sunar. Dosya işlemleri temelde ikiye ayrılır:

- ◊ Dosyanın kendisiyle ilgili işlemler (silme, taşıma, varlık kontrolü) ve
- ◊ Dosyanın içeriğiyle ilgili işlemler (okuma, yazma).

§16.4.1 Dosya ve Klasör Bilgisi

C#, statik yöntemler yardımıyla dosya sistemine hızlıca müdahale etmenizi sağlar.

1. `File.Exists(yol)`: Dosya var mı?
2. `File.Delete(yol)`: Dosyayı sil.
3. `File.Create(yol)`: Dosya Oluştur.
4. `File.Move(yol)`: Dosyayı Taşı.
5. `Directory.CreateDirectory(yol)`: Yeni klasör oluştur.
6. `Directory.GetFiles(yol, filtre, aramaseçenekleri)`: Klasördeki dosyaları öğrenmek için kullanılır

Bir Klasördeki Dosyaları Öğrenme

Klasördeki tüm dosyaları öğrenmek için aşağıdaki kodu yazarız;

```
var FileSearchRes = Directory.GetFiles(Path, "*.*", SearchOption.AllDirectories);
```

Belirtilen dizindeki tüm dosyaları temsil eden bir `FileInfo` dizisi döndürür. `*.*` ifadesindeki yıldız karakteri **joker (wildcard)** olarak adlandırılır. Bu yüzden dosya adı ve uzantısında **yıldız karakteri kullanılmaz**. Adı ve uzantısı ne olursa olsun anlamında kullanılmıştı. Pdf uzantılı dosyaları seçmek için joker `*.pdf` şeklinde yazılır. Arama seçenekleri aşağıdaki tabloda verilmiştir;

Seçenek	Kapsam	Açıklama
<code>TopDirectoryOnly</code>	Sadece Hedef Dizin	Arama işlemini sadece belirtilen klasörle sınırlar. Alt klasörlere bakmaz. (Varsayılan mod)
<code>AllDirectories</code>	Tüm Alt Dizinler	Belirtilen klasörden başlayarak hiyerarşideki tüm alt klasörleri özyinelemeli (recursive) olarak tarar.

Tablo 16.1: SearchOption Seçenekleri ve Karşılaştırması

```
using System;
using System.IO;

string yol = @"C:\Projelerim";

// 1. Sadece ana klasördeki .jpg dosyaları
string[] ustDosyalar = Directory.GetFiles(yol, "*.jpg", SearchOption.TopDirectoryOnly);

// 2. Ana klasör VE tüm alt klasörlerdeki .jpg dosyaları
string[] tumDosyalar = Directory.GetFiles(yol, "*.jpg", SearchOption.AllDirectories);

Console.WriteLine($"Ana Klasörde: {ustDosyalar.Length} dosya bulundu.");
Console.WriteLine($"Tüm Alt Klasörlerde Toplam: {tumDosyalar.Length} dosya bulundu.");

//3. Ana klasör VE tüm alt klasörlerdeki .pdf dosyaları
string[] FileSearchRes = Directory.GetFiles(yol, "*.pdf", SearchOption.AllDirectories);
foreach (var item in FileSearchRes)
    Console.WriteLine(item);
```

`SearchOption.AllDirectories` seçeneğini kullanırken, tarama yapılan alt klasörlerden herhangi birine erişim izniniz yoksa (örneğin sistem klasörleri), yöntem bir `UnauthorizedAccessException` fırlatır ve tüm arama yarıda kesilir. C#8.0+ ve sonrasında, bu sorunu aşmak için `EnumerationOptions` sınıfı kullanılabilir. Bu sınıf, erişilemeyen klasörleri "atla" (`IgnoreInaccessible`) demenize olanak tanır.

```
string yol = @"C:\Projelerim";
var options = new EnumerationOptions { IgnoreInaccessible = true, RecurseSubdirectories = true };
var dosyalar = Directory.GetFiles(yol, "*.*", options);
```

Mevcut Bir Dosyayı Taşıma

`File` statik sınıfının `Move()` yöntemini kullanarak dosyaları taşıyabiliriz. Yalnızca dosyanın klasörünü değiştirir. Bu işlem dosya boyutuna göre zaman almaz. Ayrıca aynı aldı dosya hedefte var ise kopyalama

olmaz.

```
//File.Move("kaynakYol", @"hedefYol");
File.Move(@"C:\Users\ILHANOZKAN\YeniDosya2.txt", @"D:\YeniDosya.kopya.txt");
```

Mevcut Bir Dosyayı Silme

Yol belirterek dosyayı `Delete()` yöntemiyle silebiliriz. Bu yöntem, dosya mevcut değilse istisna oluşturmaz, ancak belirtilen yol geçersizse veya çağıran gerekli izinlere sahip değilse istisna oluşturur. `Delete()` çağrılarını her zaman `try-catch` bloğunun içine koymalı ve tüm beklenen istisnaları işlemelisiniz. Birden fazla program aynı dosyayı silmeye kalkışabilir ve bu durumda yarış durumu (`race condition`) ortaya çıkar. Bunu önlemek için silme mantığını lock ifadesinin içine koymalıyız.

```
string path = @"D:\YeniDosya.kopya.txt";
File.Delete(path);
```

§16.4.2 Yol Yönetimi

Farklı işletim sistemlerinde farklı klasör ayracı; Windows işletim sisteminde ters bölü (\), Unix ve Linux işletim sisteminde (/) kullanır. Yolların bozulmaması için `Path` sınıfının `Combine()` yöntemini kullanmak profesyonel bir zorunluluktur.

```
string klasor = "Veriler";
string dosya = "ayarlar.json";
// İşletim sistemine göre en doğru yolu birleştirir:
string tamYol = Path.Combine(klasor, dosya); //Linux için "Veriler/ayarlar.json" üretirken
→ Windows için "Veriler\ayarlar.json" üretir.
```

§16.4.3 Dosya Oluşturma

`File` statik sınıfının `Create()` yöntemini kullanarak dosyalar oluşturabiliriz. Bu yöntem, verilen yolda dosyayı oluşturur, aynı zamanda dosyayı açar ve bize dosyanın `FileStream` nesnesini verir. İşimiz bittikten sonra dosyayı `Close()` yöntemi ile kapattığımızdan emin olmalıyız.

```
FileStream akış = File.Create("Metin.txt"); // Programın çalıştığı klasörde dosya oluşturur.
// Dosya İşlemleri...
akış.Close(); //Dosya Create ile açıldığından mutlaka kapatılmalıdır.
```

Buna eşdeğer olarak `Open()` yöntemini kullanabiliriz.

```
FileStream akış = File.Open("C:\\Users\\ILHANOZKAN\\Metin.txt", FileMode.Create);
// Programın çalıştığı dizine dosya oluşturur.
akış.Close();
```

Bu yöntemde ikinci parametre olarak dosya açma modunu belirten `FileMode` kullanabilir.

Mod	Dosya Varsa	Dosya Yoksa	Açıklama
<code>CreateNew</code>	Hata Fırlatır	Oluşturur	Sadece yeni bir dosya oluşturmak için kullanılır. Dosya zaten varsa risk almaz.
<code>Create</code> <code>Open</code>	Üzerine Yazar Açar	Oluşturur Hata Fırlatır	Dosya varsa içeriğini siler ve sıfırdan başlar. Var olan bir dosyayı okumak veya yazmak için açar.
<code>OpenOrCreate</code>	Açar	Oluşturur	En güvenli yoldur. Varsa açar, yoksa yeni bir tane oluşturur.
<code>Truncate</code>	İçini Boşaltır	Hata Fırlatır	Dosyayı açar ama içindeki tüm veriyi anında siler (0 byte yapar).
<code>Append</code>	Sonuna Ekler	Oluşturur	Mevcut veriye dokunmaz, yazmaya dosyanın en sonundan başlar.

Tablo 16.2: FileMode Seçenekleri ve Davranışları

```
string yol = "log.txt";
// Senaryo A: Her seferinde eskiyi sil, yeni başla
using (FileStream fs = new FileStream(yol, FileMode.Create)) {
    // ...
}
// Senaryo B: Eski verilere dokunma, altına ekle (Loglama için ideal)
using (FileStream fs = new FileStream(yol, FileMode.Append)) {
    // ...
}
```

```
}
```

`using` anahtar kelimesi, **açılan bir kaynağın kapatılmasını otomatik olarak yapar**. Böylece `Close()` yazmayı unutmaktan korkmayız.

```
using (var akış=File.Create(@"C:\Users\ILHANOZKAN\Metin.txt"))
{
    //akış kullanılmıyorsa blok dışında otomatik olarak kapatılır.
}
```

Dosya oluşturmanın bir diğer yolu da `FileStream` sınıfını kullanmaktır.

```
FileStream akış = new FileStream("Metin.txt", FileMode.OpenOrCreate, FileAccess.ReadWrite,
    ↪ FileShare.None);
//...
fs.Close();
```

Bu yöntemin fazladan `FileAccess` ve `FileShare` adlı iki parametresi daha vardır. `FileMode` genellikle bu iki iki yardımcı `enum` ile birlikte kullanılır.

1. `FileAccess`: Sizin dosyaya ne yapacağınız (`Read`, `Write`, `ReadWrite`).
2. `FileShare`: Siz dosyayı kullanırken başkalarının (başka programların) ne yapabileceği (`None`, `Read`, `Write`).

Örnek olarak dosyayı sadece okumak için aç ve ben okurken kimse değiştiremesin istiyorsak aşağıdaki gibi dosyayı açarız;

```
FileStream fs = new FileStream("data.bin", FileMode.Open, FileAccess.Read, FileShare.Read);
//...
fs.Close();
```

16.5 Metin Dosyalarını Toptan Okuma ve Yazma

Bir dosyanın tüm içeriğini bir metin (`string`) değişkenine koymak için `System.IO.File.ReadAllText()` ve bir metin değişkeni yazmak için `System.IO.File.WriteAllText()` yöntemi kullanılabilir;

```
System.IO.File.WriteAllText(@"Metin.txt", "Bu bir deneme metnidir.\nBu metin ikinci satırdır.");
string text = System.IO.File.ReadAllText(@"Metin.txt");
string[] lines = text.Split(new[] { '\n' }, StringSplitOptions.None);
int satırSayısı = lines.Length;
Console.WriteLine($"Toplam {satırSayısı} satır var.");
```

Tüm satırları bir metin dizisine (`string[]`) okumak için `System.IO.File.ReadAllLines()` ve yazmak için `System.IO.File.WriteAllLines()` yöntemi kullanılabilir;

```
string[] lines = System.IO.File.ReadAllLines(@"\Metin.txt");
foreach (string line in lines)
{
    Console.WriteLine(line);
}
```

§16.5.1 Büyük Metin Dosyaları

Belleği çok zorlayacak büyük metin dosyalarıyla çalışırken, bir dosyadaki tüm satırları `IEnumerable<string>` jenerik ara yüzüne okumak için `System.IO.File.ReadLines()` yöntemi kullanabiliriz;

```
using System.Text; //Encoding için gerekli
IEnumerable<string> AllLines = File.ReadLines(@"\Metin.txt", Encoding.Default);
foreach (string line in AllLines)
{
    Console.WriteLine(line);
}
```

Bu yöntemle farklı yerel kodlamaya (`encoding`) sahip metin dosyasının okuma imkanına da sahip oluruz. Ayrıca `File.WriteLine()` yöntemi yazmak için de kullanılabilir.

16.6 Akışlar ile Dosya Yazma ve Okuma

Dosya içeriğiyle uğraşırken en güvenli ve yaygın yol `StreamWriter` ve `StreamReader` sınıflarını kullanmaktır. Bu sınıflar, karmaşık bayt işlemlerini bizim yerimize yöneterek doğrudan **metinlerle** (`string`) çalışmamıza

olarak tanır.

Akışlar verinin nerede saklandığıyla ilgilenmez. Bu teorik olarak doğru olsa da, `FileStream` alt sınıfının doğrudan saklama ortamına bağlı olduğunu, ancak `Stream` ebeveyn sınıfı bu soyutlamayı sağlar.

§16.6.1 Dosyaya Yazma

```
string yol = "metin.txt";
using (StreamWriter sw = new StreamWriter(yol)) // 'using' bloğu, işlem bitince dosyayı otomatik
    ↪ kapatır.
{
    sw.WriteLine("C# Programlama Kitabı");
    sw.WriteLine("Bölüm: Dosya İşlemleri");
    sw.Write("Tarih: ");
    sw.WriteLine(DateTime.Now);
} // sw.Dispose() burada otomatik çağrılır.
```

§16.6.2 Dosyadan Okuma

```
string yol = "metin.txt";
if (File.Exists(yol))
{
    using (StreamReader sr = new StreamReader(yol))
    {
        string satır;
        while ((satır = sr.ReadLine()) != null) // akıştan bir satır oku
        {
            Console.WriteLine($"Okunan: {satır}");
        }
    }
}
```

§16.6.3 İkili Dosyalar

Metin dosyalarından farklı olarak, verileri bellekte tutulduğu gibi ikili olarak dosyalara yazabilir ve okuyabiliriz. Bu şekilde oluşan dosyalar gözle okunamazlar. Çünkü işlemcinin doğal olarak anladığı ikili sayı sisteminde veriler dosyalara yazılır ve okunur.

```
using System.Text;
string yol = "ikili.bin";

//İKİLİ OLARAK DOSYAYA YAZMA:
using (var stream = File.Open(yol, FileMode.Create))
{
    using (var writer = new BinaryWriter(stream, Encoding.UTF8, false))
    {
        writer.Write(1.250F);
        writer.Write(@"c:\Temp");
        writer.Write("İlhan Özkan");
        writer.Write(10);
        writer.Write(true);
    }
}

//İKİLİ OLARAK DOSYADAN OKUMA:
float oran;
string geçiciKlasör;
string geçiciKullanıcıAdı;
int adet;
bool doluMu;
if (File.Exists(yol))
{
    using (var stream = File.Open(yol, FileMode.Open))
    {
        using (var reader = new BinaryReader(stream, Encoding.UTF8, false))
        {
```

```

        oran = reader.ReadSingle();
        geçiciKlasör = reader.ReadString();
        geçiciKullanıcıAdı = reader.ReadString();
        adet = reader.ReadInt32();
        doluMu = reader.ReadBoolean();
    }
}
//Okunanları Konsola Yazma
Console.WriteLine($"Oran: {oran}");
Console.WriteLine($"Geçici Klasör: {geçiciKlasör}");
Console.WriteLine($"Kullanıcı Adı: {geçiciKullanıcıAdı}");
Console.WriteLine($"Adet: {adet}");
Console.WriteLine($"Dolu Mu?: {doluMu}");
}

```

Oluşan **ikili.bin** dosyasına;

1. İlk önce kayan noktalı sayı (**float**) değeri,
2. Sonra iki adet metin (**string**)
3. Ardından bir tamsayı (**int**) ve
4. Son olarak **bool** değer ikili olarak yazılmıştır.

Bellekte saklandığı şekliyle dosyaya kaydedildiğinden içeriği gözle okunamaz. İkili dosyaları açan görüntüleyiciler ile açılıp görülebilir. Bir ikili dosyayı onaltılık rakamlara çevirip metin olarak görüntülemek için Windows ortamında `certutil -encodehex ikili.bin ikili.txt` komutu verilebilir. Benzer işlem Linux ortamında `hexdump -C ikili.bin | less >> ikili.txt` komutuyla yapılabilir. Böylece onaltılık tabanda sayılar ile dosyanın her bir baytı **ikili.txt** dosyasına yazılacaktır. Artık dönüştürülen dosyayı istediğimiz metin düzenleyicisinde açıp okuyabiliriz.

```

0000 00 00 a0 3f 07 63 3a 5c  54 65 6d 70 0d c4 b0 6c  ...?.c:\Temp...l
0010 68 61 6e 20 c3 96 7a 6b  61 6e 0a 00 00 00 01    han ..zkan....

```

16.7 Bellek Akışları

Dosya işlemleri veriyi diskte saklarken, **MemoryStream** veriyi RAM (Bellek) üzerinde bir akış (**stream**) olarak yönetmenizi sağlar. **MemoryStream**, verileri bayt dizileri (**byte[]**) olarak bellekte tutan bir **Stream** türevidir. Fiziksel bir dosya ile çalışıyormuş gibi kod yazmanıza rağmen, tüm işlemler RAM üzerinde gerçekleştiği için inanılmaz hızlıdır.

Akışlar verinin nerede saklandığıyla ilgilenmez. Bu teorik olarak doğru olsa da, **FileStream** gibi **MemoryStream** alt sınıflarının doğrudan saklama ortamına bağlı olduğunu, ancak **Stream** ebeveyn sınıfı bu soyutlamayı sağlar.

Neden MemoryStream Kullanılır?

- ♦ **Geçici Depolama:** Veriyi diske kaydedip silmekle uğraşmadan bellekte manipüle etmek için.
- ♦ **Veri Dönüşümleri:** Bir resmi veya nesneyi **serileştirme** (**serialization**) yapıp ağ üzerinden göndermeden önce paketlemek için.
- ♦ **Şifreleme:** Veriyi şifrelerken veya çözerken ara katman olarak.

Aşağıda bir metin (**string**) ifadesinin nasıl belleğe yazıldığını ve oradan nasıl geri okunduğunu gösterir:

```

using System.Text; //Encoding sınıfı için

string mesaj = "C# Kitabı MemoryStream Örneği";
byte[] veriPaketi = Encoding.UTF8.GetBytes(mesaj);

// Bellekte bir akış oluşturuyoruz
using (MemoryStream ms = new MemoryStream())
{
    // 1. Veriyi belleğe yaz
    ms.Write(veriPaketi, 0, veriPaketi.Length);

    // DİKKAT: Yazma işleminden sonra imleç sona gider.
    // Okumak için imleci başa sarmalıyız!
    ms.Position = 0;

    // 2. Bellekten oku
    byte[] okunanVeri = new byte[ms.Length];
}

```



```
ms.Read(okunanVeri, 0, okunanVeri.Length);

string sonuc = Encoding.UTF8.GetString(okunanVeri);
Console.WriteLine("Bellekten Okunan: " + sonuc);
}
```

16.8 Serileştirme ve Seriden Çıkarma

Serileştirme (serialization), bir nesnenin durumunu (verilerini) saklamak veya bir ağ üzerinden iletmek amacıyla bayt dizisine, JSON'a veya XML formatına dönüştürülmesi işlemidir. **Seriden çıkarma (deserialization)** ise bu serileştirilmiş verinin tekrar canlı bir nesneye dönüştürülmesidir. Serileştirilen veri, bir akış (**stream**) üzerinden gönderilir veya alınır.

§16.8.1 JSON Serileştirme

JSON (**J**avascript **O**bject **N**otation), günümüzde en çok kullanılan yöntemdir. Modern .NET projelerinde `System.Text.Json` kütüphanesi standart olarak gelir.

```
using System;
using System.Text.Json;

public class Kitap
{
    public string Ad { get; set; }
    public string Yazar { get; set; }
    public int Yil { get; set; }
}

public class Program
{
    public static void Main()
    {
        var kitap = new Kitap { Ad = "C# Rehberi", Yazar = "İlhan", Yil = 2026 };
        // 1. Serileştirme (Nesne -> JSON String)
        string jsonString = JsonSerializer.Serialize(kitap);
        Console.WriteLine($"JSON Çıktısı: {jsonString}"); //JSON Çıktısı: {"Ad":"C#
        ↪ Rehberi","Yazar":"\u0130lhan","Yil":2026}
        // 2. Ters Serileştirme (JSON String -> Nesne)
        Kitap cozulenKitap = JsonSerializer.Deserialize<Kitap>(jsonString);
        Console.WriteLine($"Çözülen Kitap: {cozulenKitap.Ad} by {cozulenKitap.Yazar}"); //Çözülen
        ↪ Kitap: C# Rehberi by İlhan
    }
}
```

`System.Text.Json` kütüphanesinin varsayılan olarak büyük-küçük harf duyarlı (**case-sensitive**) olduğunu, bu nedenle serileştirme seçeneklerinde (`JsonSerializerOptions`) `PropertyNameCaseInsensitive = true` ayarının sıklıkla yapılması gerekir.

§16.8.2 XML Serileştirme

XML (**eX**tensible **M**arkup **L**anguage), daha çok konfigürasyon dosyalarında ve eski sistemlerle entegrasyonda tercih edilir.

```
using System.Xml.Serialization;
using System.IO;

public class Kitap
{
    public string Ad { get; set; }
    public string Yazar { get; set; }
    public int Yil { get; set; }
}

public class Program
{
    public static void Main()
```

```

{
    var kitap = new Kitap { Ad = "C# Rehberi", Yazar = "İlhan", Yil = 2026 };
    // XML serileştirme için boş bir yapıcı (constructor) gereklidir.
    XmlSerializer serializer = new XmlSerializer(typeof(Kitap));
    using (StringWriter writer = new StringWriter())
    {
        serializer.Serialize(writer, kitap);
        string xmlOutput = writer.ToString();
        Console.WriteLine(xmlOutput);
    }
}
}
/*Program Çıktısı:
<?xml version="1.0" encoding="utf-16"?>
<Kitap xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <Ad>C# Rehberi</Ad>
  <Yazar>İlhan</Yazar>
  <Yil>2026</Yil>
</Kitap>
*/

```

§16.8.3 Binary (İkili) Serileştirme

Nesneyi doğrudan makine diline yakın bir bayt dizisine dönüştürür. Hızlıdır ve az yer kaplar ancak okunabilir değildir. C#11+ sürümlerinde güvenlik nedeniyle **BinaryFormatter** yerine **Protocol Buffers** (kısaca **protobuf**) veya **MessagePack** gibi kütüphaneler önerilir.

§16.8.4 Serileştirme Kontrol Mekanizmaları

Serileştirme süreci aşağıdaki özniteliklerle (**attributes**) manipüle edebilir;

- ◊ **[JsonPropertyName("book_name")]**: JSON içindeki anahtar ismini değiştirir.
- ◊ **[JsonIgnore]**: Belirli bir alanın serileştirilmesini engeller (örneğin şifreler).
- ◊ **[Serializable]**: Bir sınıfın ikili (**binary**) serileştirmeye uygun olduğunu işaretler.

§16.8.5 Özet

Format	Okunabilirlik	Boyut	Kullanım Alanı
JSON	Çok Yüksek	Orta	Web API, NoSQL, Mobil.
XML	Yüksek	Büyük	Konfigürasyon, SOAP servisler.
Binary	Yok	Çok Küçük	Yüksek performanslı veri transferi.
Protobuf	Yok	En Küçük	Mikroservisler (gRPC).

Tablo 16.3: Serileştirme Formatlarının Teknik Analizi

16.9 Düşün ve Çöz

- ◊ **Düşün:** Akış (**stream**) soyutlamasının temel değeri nedir? Ağ üzerinden veri okumak ile dosyadan veri okumak aynı API ile yapılabiliyorsa programcıya ne kazandırır? Bu soyutlama olmadan aynı işi yapmak nasıl daha karmaşık olurdu?
- ◊ **Düşün:** **FileStream**, **StreamReader** ve **File.ReadAllText()** arasındaki fark nedir? Büyük dosyalarda satır satır okuma (**StreamReader**) yerine tüm dosyayı belleğe yüklemenin (**File.ReadAllText**) riskleri nelerdir?
- ◊ **Düşün:** Akışların mutlaka kapatılması gerektiğini biliyoruz. **using** bloğu ile **try-finally** bloğunun davranışı hangi senaryoda farklılaşır? C#8 ile gelen **using** bildirimi (**using declaration**) bu sorunu nasıl sadeleştirir?
- ◊ **Düşün:** Serileştirme (**serialization**) ne anlama gelir? JSON, XML ve ikili (**binary**) serileştirmenin avantaj ve dezavantajlarını karşılaştırınız. Hangi senaryoda hangisi tercih edilir?
- ◊ **Düşün:** **System.Text.Json** ve **Newtonsoft.Json** arasındaki temel farklar nelerdir? .NET 5+ ile birlikte **System.Text.Json**'ın varsayılan kütüphane haline gelmesinin nedenleri nelerdir?
- ◊ **Düşün:** Dosya yollarını (**path**) hardcode yazmak neden kötü bir pratiktir? **Path.Combine()**, **Environment.GetFolderPath()** ve **AppDomain.CurrentDomain.BaseDirectory** yöntemlerinin doğru kullanımı çapraz platform uyumluluğunu nasıl sağlar?
- ◊ **Düşün:** Dosya izleme (**FileSystemWatcher**) ne gibi kullanım senaryolarına hizmet eder? Bir dizini

izlerken ortaya çıkabilecek yarış koşulları (**race condition**) nasıl ele alınır?

- ◇ **Çöz:** Bir öğrenci listesini JSON formatında dosyaya kaydeden ve dosyadan geri okuyan program yazınız. `System.Text.Json` kullanınız; kayıt bulunamazsa boş liste dönsün, varsa listeye yeni kayıt eklensin.
- ◇ **Çöz:** Büyük bir metin dosyasını satır satır okuyarak her satırdaki sözcük sayısını, toplam satır sayısını ve en uzun satırı bulan; sonuçları ayrı bir rapor dosyasına yazan program yazınız. `StreamReader` kullanınız.
- ◇ **Çöz:** Bir dizin altındaki tüm `.txt` dosyalarını özyinelemeli olarak tarayan, her dosyanın adını, boyutunu ve son değiştirilme tarihini listeleyen; sonuçları büyükten küçüğe boyuta göre sıralayan program yazınız.
- ◇ **Çöz:** Bir yapılandırma (**config**) sistemi tasarlayınız: anahtar-değer çiftlerini JSON dosyasına kaydeden, okuyan ve güncelleyen; dosya yoksa varsayılan değerlerle oluşturan bir `ConfigManager` sınıfı yazınız.
- ◇ **Çöz:** `FileSystemWatcher` kullanarak belirli bir klasörü izleyen; yeni dosya eklendiğinde, silindiğinde veya değiştirildiğinde konsola zaman damgalı bildirim yazan bir program tasarlayınız.